K8S-06: Namespace Organization and Boundaries

> **Series:** K8S — Kubernetes Monitoring | **Notebook:** 6 of 14 | **Created:** January 2026 | **Last Updated:** 10/13/2026

Organizing Kubernetes Monitoring with Namespaces

Namespaces provide logical boundaries in Kubernetes for resource isolation, access control, and organizational structure. This notebook covers namespace strategies and how to leverage them in Dynatrace for filtered views, access control, and cost allocation.

Table of Contents

1. Namespace Strategies
2. Namespace Monitoring in Dynatrace
3. Resource Quotas and Limits
4. Namespace-Based Access Control
5. Multi-Tenant Clusters
6. Cost Allocation by Namespace
7. Namespace Best Practices

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Kubernetes monitoring
DynaKube	Deployed with namespace selector configured
Permissions	`storage:metrics:read`, `storage:smartscape:read`
Knowledge	K8S-01 Fundamentals, K8S-05 Workload Monitoring

1. Namespace Strategies

Common Namespace Patterns

Strategy	Structure	Use Case
By Environment	`dev`, `staging`, `prod`	Single cluster, multiple envs
By Team	`team-checkout`, `team-catalog`	Team ownership
By Application	`app-frontend`, `app-backend`	Application isolation
By Function	`monitoring`, `logging`, `ingress`	Infrastructure services
Hybrid	`team-checkout-prod`	Combined approach

Namespace Naming Conventions

```
&lt;prefix&gt;-&lt;owner&gt;-&lt;env&gt;

Examples:
app-checkout-prod
team-platform-dev
infra-monitoring
```

System Namespaces

Namespace	Purpose	Monitor?
`kube-system`	Core K8s components	Yes (critical)
`kube-public`	Public resources	Minimal
`kube-node-lease`	Node heartbeats	No
`default`	Catch-all	Discourage use

```
// List all Kubernetes namespaces (smartscape topology)
smartscapeNodes "K8S_NAMESPACE"
| fields entity.name = name, tags
| sort entity.name asc
| limit 50

// Legacy alternative (deprecated for new content):
// fetch dt.entity.cloud_application_namespace
// | fields entity.name, tags
// | sort entity.name asc
// | limit 50
```

2. Namespace Monitoring in Dynatrace

DynaKube Namespace Selector

Control which namespaces receive OneAgent injection:

```
spec:
  oneAgent:
    cloudNativeFullStack:        # or applicationMonitoring — the selector
sits inside the mode
      namespaceSelector:
        matchLabels:
          monitoring: dynatrace
```

Or exclude specific namespaces:

```
spec:
  oneAgent:
    cloudNativeFullStack:
      namespaceSelector:
        matchExpressions:
          - key: monitoring
            operator: NotIn
            values:
              - disabled
```

`namespaceSelector` is not a top-level `spec` field in any DynaKube API version (checked against the Operator 1.11.0 CRD schema). It belongs under the injection mode (`spec.oneAgent.cloudNativeFullStack` or `spec.oneAgent.applicationMonitoring`), and there is a separate one under `spec.metadataEnrichment` for enrichment.

Namespace Labels for Filtering

Label	Purpose	Example
`team`	Team ownership	`team: checkout`
`env`	Environment	`env: production`
`cost-center`	Billing allocation	`cost-center: eng-123`
`monitoring`	Injection control	`monitoring: enabled`

```
// Namespace CPU usage — sum() adds up every container in the namespace; avg() would
// return the average container's usage, which ranks namespaces by container size, not consumption
// rollup: avg averages each container within a time bucket before the cross-container sum;
// without it, sum() also adds the 1-min points in each bucket and wide timeframes read ~10x high.
timeseries cpuMillicores = sum(dt.kubernetes.container.cpu_usage, rollup: avg), from:-1h, by:{k8s.namespace.name}
| fieldsAdd avgCpuMillicores = arrayAvg(cpuMillicores)
| sort avgCpuMillicores desc
| limit 15
```

```
// Memory usage by namespace — sum() of every container's working set in the namespace
// rollup: avg averages each container within a time bucket before the cross-container sum;
// without it, sum() also adds the 1-min points in each bucket and wide timeframes read ~10x high.
timeseries memBytes = sum(dt.kubernetes.container.memory_working_set, rollup: avg), from:-1h, by:{k8s.namespace.name}
| fieldsAdd avgMemBytes = arrayAvg(memBytes)
| sort avgMemBytes desc
| limit 15
```

Per-Namespace Service Detection Scoping

Service detection settings — including **Enhanced Endpoints for SDv1** (Dynatrace v1.329+) — can be overridden per Kubernetes namespace, not just at the environment level. Useful when one namespace runs services that need different endpoint-naming behavior than the tenant default.

Override path: *Kubernetes app → select cluster or namespace → Actions menu → Service detection settings → Process and contextualize → Services → Enhanced endpoints for SDv1*

Scopes the setting can be set at — the docs list *"the entire environment, a specific host group, or a Kubernetes namespace and cluster"*:

Scope	When to use
Environment-wide	Default policy for the whole tenant
Host group	Group of hosts with shared service-detection needs
Kubernetes cluster	All namespaces in one cluster (e.g., a multi-tenant cluster owned by one team)
Kubernetes namespace	One workload — typically when a service behind a reverse proxy (Nginx/Apache/IIS) collapses to `GET /*` and you want to keep the per-endpoint metrics elsewhere

Tenant-creation-date reminder: environments created at v1.333+ have Enhanced Endpoints always on and not configurable at any scope. Per-namespace overrides apply only to older environments where the setting is toggleable.

> **Sources:** Enhanced endpoints for SDv1 (DT docs).

3. Resource Quotas and Limits

ResourceQuota Monitoring

ResourceQuotas limit total resource consumption per namespace:

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: compute-quota
  namespace: team-checkout
spec:
  hard:
    requests.cpu: "10"
    requests.memory: 20Gi
    limits.cpu: "20"
    limits.memory: 40Gi
    pods: "50"
```

LimitRange for Defaults

```
apiVersion: v1
kind: LimitRange
metadata:
  name: default-limits
  namespace: team-checkout
spec:
  limits:
    - default:
        cpu: "500m"
        memory: "512Mi"
      defaultRequest:
        cpu: "100m"
        memory: "128Mi"
      type: Container
```

Quota Utilization Metrics

Metric	Grail key	Alert When
CPU quota usage	`dt.kubernetes.container.requests_cpu` (sum by namespace)	>80% of quota
Memory quota usage	`dt.kubernetes.container.requests_memory` (sum by namespace)	>80% of quota
Pod count	`dt.kubernetes.pods`	Approaching limit

> **Requests are container-grain.** A ResourceQuota is enforced per namespace, but Dynatrace emits requests per **container** — there is no `dt.kubernetes.workload.requests_cpu` or `.requests_memory`. Sum the `dt.kubernetes.container.*` series across `k8s.namespace.name` to get the figure the quota is measured against. Confirm what your tenant carries with `metrics | filter startsWith(metric.key, "dt.kubernetes") | summarize n = count(), by: {metric.key}` — note `metrics` takes `from:` with **no leading comma**.
>
> **Quota tracking across ActiveGate 1.343 / 1.345 upgrades:** request metrics carry an effective pod value — init containers are accounted for from ActiveGate 1.343, and pod-level `spec.resources` (Kubernetes 1.34+) and pod `overhead` from 1.345 — so reserved-CPU and reserved-memory figures can step at either upgrade with no workload change. Read a trend spanning either boundary as **two series, not one**, and re-baseline any >80%-of-quota alert afterward. Full explanation: K8S-08 §2.

```
// CPU requests by namespace (quota tracking)
// Requests are emitted at container grain — there is no
dt.kubernetes.workload.requests_cpu.
// Sum across containers to get the namespace reservation a ResourceQuota is
measured against.
// rollup: avg averages each container within a time bucket before the cross-
container sum;
// without it, sum() also adds the 1-min points in each bucket and wide
timeframes read ~10x high.
timeseries cpuReq = sum(dt.kubernetes.container.requests_cpu, rollup: avg),
from:-1h, by:{k8s.namespace.name}
| fieldsAdd avgReqMillicores = round(arrayAvg(cpuReq), decimals: 0)
| fields k8s.namespace.name, avgReqMillicores
| sort avgReqMillicores desc
| limit 15
```

```
// Memory requests by namespace (GiB, quota tracking)
// rollup: avg averages each container within a time bucket before the cross-
container sum;
// without it, sum() also adds the 1-min points in each bucket and wide
timeframes read ~10x high.
timeseries memReq = sum(dt.kubernetes.container.requests_memory, rollup: avg),
from:-1h, by:{k8s.namespace.name}
| fieldsAdd avgReqGiB = round(arrayAvg(memReq) / 1073741824, decimals: 2)
| fields k8s.namespace.name, avgReqGiB
| sort avgReqGiB desc
| limit 15
```

4. Namespace-Based Access Control

Dynatrace IAM: policies with a namespace condition

Grail record permissions take a `WHERE` condition on `storage:k8s.namespace.name`, so a policy can grant read access to one namespace's data:

```
ALLOW storage:logs:read, storage:spans:read, storage:events:read,
storage:metrics:read
  WHERE storage:k8s.namespace.name = "checkout";
ALLOW storage:buckets:read
  WHERE storage:table-name IN ("logs", "spans", "events", "metrics");
```

The condition supports `=`, `IN`, `startsWith` and `MATCH`. To reuse one policy across teams, keep the permission in the policy and move the condition into a **boundary** bound with the group (IAM series).

Two details decide whether this grants anything:

- **The bucket permission.** `storage:buckets:read` is *"Required additionally to a table permission"* — without the second statement the first reads nothing.
- **Arrays on events.** *"Using `=`, `STARTSWITH` or `IN` when the field holds an array will always return `false`."* On the validation tenant (10/13/2026) about 7% of events that carried `k8s.namespace.name` held it as an array, so an `=` condition silently hides those events. Where events matter, use `MATCH ("checkout")` instead.

Segment by Namespace

Segments filter what a view shows; they do not restrict access.

Segment	Include condition on `k8s.namespace.name`	Use Case
`seg-checkout`	equals `checkout`	Team view
`seg-production`	ends with `-prod` (a `*` wildcard in the value)	Prod only
`seg-platform`	is one of `monitoring`, `logging`	Platform team

Segment filter syntax and the operators each data type accepts are covered in ORGNZ-10 §1.

Kubernetes RBAC Alignment

Mirror the Kubernetes roles you already bind per namespace:

Kubernetes binding	Dynatrace equivalent
`admin` / `edit` ClusterRole bound in a namespace	Read policy with that namespace's condition, plus write permissions the team needs (for example settings in its own scope)
`view` ClusterRole bound in a namespace	Read policy with that namespace's condition
`cluster-admin`	Read policy without a namespace condition

> **Sources:** IAM policy statements (DT docs) — *"ALLOW storage:events:read WHERE storage:k8s.namespace.name = "production""*, and `storage:buckets:read` *"Required additionally to a table permission"*; Permissions in Grail (DT docs).

5. Multi-Tenant Clusters

Tenant Isolation Strategies

Level	Mechanism	Dynatrace Support
Soft	Namespace + NetworkPolicy	IAM namespace conditions, segments
Hard	Separate clusters	One DynaKube per cluster, each with its own environment

Separate DynaKubes per namespace group

The Operator's own `multipleDynakubes` sample runs two DynaKubes in one cluster, each scoped by a `namespaceSelector` inside its mode. To send different namespaces to different environments, give each DynaKube its own `apiUrl` and use **`applicationMonitoring`**, which injects code modules into pods without deploying a host OneAgent:

```
# Tenant A
apiVersion: dynatrace.com/v1beta6
kind: DynaKube
metadata:
  name: dynakube-tenant-a
  namespace: dynatrace
spec:
  apiUrl: https://tenant-a.live.dynatrace.com/api
  oneAgent:
    applicationMonitoring:
      namespaceSelector:
        matchLabels:
          tenant: a
```

Keep the selectors from overlapping, so no namespace matches two DynaKubes. Do not give two DynaKubes a host-level mode (`cloudNativeFullStack`, `hostMonitoring`, `classicFullStack`) on the same nodes: that puts two OneAgents on one host, and Dynatrace documents *"A single OneAgent per host is required to collect all relevant monitoring data"*.

Shared DynaKube (Soft Isolation)

Single DynaKube with access controlled in IAM:

```
spec:
  oneAgent:
    cloudNativeFullStack: {}
  # All namespaces monitored, access controlled via IAM namespace conditions
```

> **Sources:** multipleDynakubes.yaml (Dynatrace GitHub), Dynatrace OneAgent (DT docs) — *"A single OneAgent per host is required to collect all relevant monitoring data—even if your hosts are deployed within Docker containers, microservices architectures, or cloud-based infrastructure."*

```
// Workload count by namespace and kind — every workload kind, not only
Deployments
// dt.kubernetes.workloads is a gauge per (cluster, namespace, kind); sum with
rollup: avg adds
// them up without also adding the points inside each time bucket. Executed
10/02/2026 —
// the deployment total matched smartscapeNodes "K8S_DEPLOYMENT" (110 = 110).
timeseries n = sum(dt.kubernetes.workloads, rollup: avg), from:-1h,
  by:{k8s.cluster.name, k8s.namespace.name, k8s.workload.kind}
| fieldsAdd workloads = round(arrayAvg(n), decimals: 0)
| fields k8s.cluster.name, k8s.namespace.name, k8s.workload.kind, workloads
| sort workloads desc
| limit 50
```

6. Cost Allocation by Namespace

Resource-Based Cost Allocation

Calculate costs based on resource consumption:

Resource	Cost Metric	Calculation
CPU	Core-hours	CPU usage × hours × rate
Memory	GB-hours	Memory usage × hours × rate
Storage	GB-hours	PVC size × hours × rate

Labels for Cost Centers

```
apiVersion: v1
kind: Namespace
metadata:
  name: team-checkout
  labels:
    cost-center: "eng-checkout-123"
    team: "checkout"
    budget-owner: "alice@example.com"
```

```
// Resource consumption by namespace (for cost allocation) — sum() of every container in the
// namespace; a per-container avg() under-states namespaces that run many modest containers
// rollup: avg averages each container within a time bucket before the cross-container sum;
// without it, sum() also adds the 1-min points in each bucket and wide timeframes read ~10x high.
timeseries cpuMillicores = sum(dt.kubernetes.container.cpu_usage, rollup: avg), from:-1h, by:{k8s.namespace.name}
| fieldsAdd avgCpuMillicores = arrayAvg(cpuMillicores)
| sort avgCpuMillicores desc
| limit 15
```

7. Namespace Best Practices

Naming and Organization

Practice	Reason
Use consistent naming	Easier filtering and automation

Practice	Reason
Apply standard labels	Cost allocation, access control
Avoid `default` namespace	Encourages explicit organization
Document ownership	Clear responsibility

Resource Management

Practice	Reason
Set ResourceQuotas	Prevent resource hogging
Configure LimitRanges	Ensure defaults
Monitor quota usage	Proactive capacity planning

Security

Practice	Reason
Apply NetworkPolicies	Namespace isolation
Use RBAC per namespace	Least privilege
Separate sensitive workloads	Compliance requirements

Monitoring

Practice	Reason
Label namespaces for filtering	Easy DQL queries
Create namespace-based dashboards	Team-specific views
Set up namespace alerts	Targeted notifications

Next Steps

With namespace organization in place, proceed to:

Next Notebook	Topic
K8S-07: Events and Logs	Kubernetes event analysis
K8S-08: DQL for Kubernetes	Advanced query patterns
K8S-09: Troubleshooting	Debugging K8s monitoring

Summary

In this notebook, you learned:

- Namespace strategies and naming conventions
- DynaKube namespace selector configuration
- Resource quota and limit monitoring
- Namespace-based access control with boundaries
- Multi-tenant cluster patterns
- Cost allocation by namespace
- Namespace best practices

References

- Set up Dynatrace on Kubernetes (DT docs)
- Kubernetes app — namespaces view (DT docs)
- DynaKube parameters (DT docs)
- Manage user permissions / boundaries (DT docs)
- Kubernetes Namespaces (kubernetes.io)
- Resource Quotas (kubernetes.io)
- smartscapeNodes command (DT docs)
- Effective pod resources (DT docs)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.